

CSE 144 Final Project Report

Transfer Learning Challenge

Group Members

Reva A (CruzID: reagarwa)

Leo Li (CruzID: bli312)

Dan Pham (CruzID: dpham26)

1. Introduction

a. Problem Goal and setting

The goal of this project is to build an image classifier for the transfer learning challenge. The task is a 100-class classification problem where each image belongs to one of the class labels 0 through 99. The training data is organized into labeled folders, while the Kaggle test set contains unlabeled images that require predicted class labels in a submission.csv file.

b. Why transfer learning is appropriate.

There are approximately 1,000 training samples of 100 categories, which is about 10 samples for each category. The data volume is small and there are a large number of categories. Training a CNN from scratch is prone to underfitting and poor generalization.

Using pre-trained visual models on large-scale data such as ImageNet/ WebLI, common visual features can be reused, and only a few parameters need to be fine-tuned to adapt to new tasks.

- c. Brief summary of your approach and main result.

We compared two transfer learning schemes: (1) EfficientNet-B0 + two-stage fine-tuning (final.ipynb); (2) SigLIP2 ViT + LoRA fine-tuning (siglipmodel.ipynb).

The latter achieved an accuracy rate of 96.363% on the validation set, significantly better than the former (61.818%). Therefore, we adopted the SigLIP2 scheme as the final submission.

2. Dataset

- a. Number of classes and dataset sizes (train/val/test).

100 classes, 1081 training cases in total,

919 train cases

162 validation cases (15% of the full dataset)

1036 test cases (given by Kaggle)

- b. Directory structure and label mapping details (ensure labels 0–99 match folders).

- i. final.ipynb

In this file, the notebook corrects the ImageFolder label mapping during dataset creation. It creates an ImageFolder dataset to inspect PyTorch's default class_to_idx mapping. Then it builds a reverse mapping from PyTorch's internal index back to the folder name:

```
idx_to_class = {v: k for k, v in full_train_bad.class_to_idx.items()}  
target_transform = lambda y: int(idx_to_class[y])
```

This means that when ImageFolder returns an internal label index, the target_transform converts it back into the true folder label. The corrected dataset is then created using:

```
full_train = datasets.ImageFolder(
    root=train_dir,
    transform=train_transforms,
    target_transform=target_transform,
)
```

ii. siglipmodel.ipynb

The notebook handles label mapping at prediction time. It first loads the dataset with ImageFolder, which creates PyTorch's default internal class indices. Then it creates dictionaries that map the internal indices back to the actual folder names:

```
idx_to_foldername = {v: k for k, v in full_dataset.class_to_idx.items()}
idx_to_true_class = {idx: int(foldername) for idx, foldername in idx_to_foldername.items()}
```

During training, the model learns using the internal ImageFolder indices. During test prediction, the model's predicted internal index is converted back to the true folder label:

```
mapped_pred = output.argmax(dim=1).item()
original_class_id = idx_to_true_class[mapped_pred]
```

The final submission.csv stores original_class_id, ensuring that the predictions match the required label IDs from 0 to 99.

c. Preprocessing (resize, normalization) and data augmentation used.

i. final.ipynb

Resize: 224 x 224

Normalize:

Mean: (0.485, 0.456, 0.406)

Std: (0.229, 0.224, 0.225)

Train augmentation:

transforms.RandomHorizontalFlip(p=0.5),

transforms.RandomRotation(15),

Validation and test:

Only resize and normalize

ii. siglipmodel.ipynb

3. Implementation

a. Model

i. Pretrained backbone used (e.g., ResNet/EfficientNet/ViT) and why.

SigLip2 ViT-SO400M-14-SigLIP2-378, which is an improved version of CLIP and more appropriate for small sample sizes of multiple classes. Compared with EfficientNet-B0 (ImageNet classification pre-training), ViT has a larger capacity and richer semantics.

- ii. Architecture changes (classifier head, pooling, dropout, etc.).

Custom L2-normalized cosine classification head: After L2 normalization of features and weights, calculate the cosine similarity, and then multiply it by the learnable scale (initial 20), which is more compatible with the SigLIP embedding space.

Feature dimension 1152 $\rightarrow$ 100 categories.

- iii. Fine-tuning strategy (frozen layers, unfreezing schedule).

Only fine-tune the last 4 Transformer blocks using LoraConfig (peft) (
r=16, lora_alpha=32, target_modules=target_modules, lora_dropout=0.05,
bias="none")

b. Training

- i. Loss function and optimizer.

- 1. Loss

CrossEntropyLoss(weight=class_weights_tensor).

Weighted inversely by category frequency to alleviate class imbalance

- 2. Optimizer

torch.optim.AdamW(filter(lambda p: p.requires_grad,
model.parameters()), lr=5e-4, weight_decay=1e-2)

- ii. Learning rate, scheduler, batch size, epochs, weight decay.

Parameters	SigLip2	EfficientNet
Batch size	32	128
LR	5e-4	Phase 1: 1e-3 Phase 2: 1e-4
Weight decay	1e-2	Phase 1: 1e-4 Phase 2: 1e-4
Epochs	20	Phase 1: 10 Phase 2: 20
Scheduler	CosineAnnealingLR, T_max=20	None
Loss	Weighted	Standard

iii. Hardware/software environment (GPU, PyTorch version).

1. final.ipynb: cpu
2. siglipmodel.ipynb: cuda gpu

4. Experiments

a. Baseline setup.

Model 1 (final.ipynb): EfficientNet-B0 (ImageNet pretrained). Phase 1: Freeze features, train only new Linear classification headers (100 classes), 10 epochs. Phase 2: Unfreeze all parameters and train for another 20 epochs.

b. Hyperparameter tuning plan and validation method.

Fix 15% validation split (seed=42). The main comparison is between backbone (EfficientNet vs SigLIP2) and fine-tuning strategies (full thawing vs LoRA). Select the best checkpoint using validation accuracy.

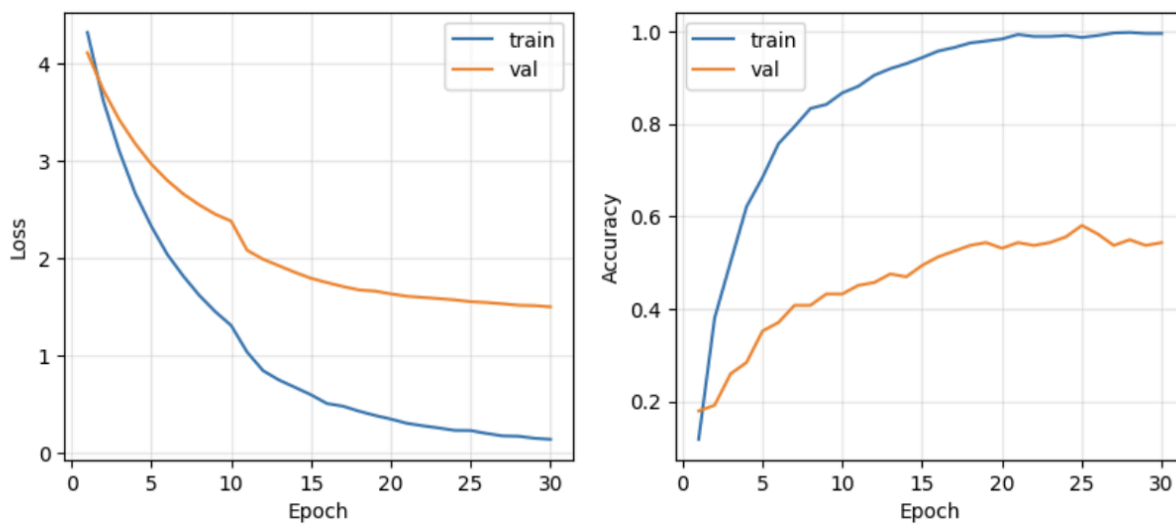
- c. Ablations (augmentations, model size, freezing strategy, LR, etc.).

See above.

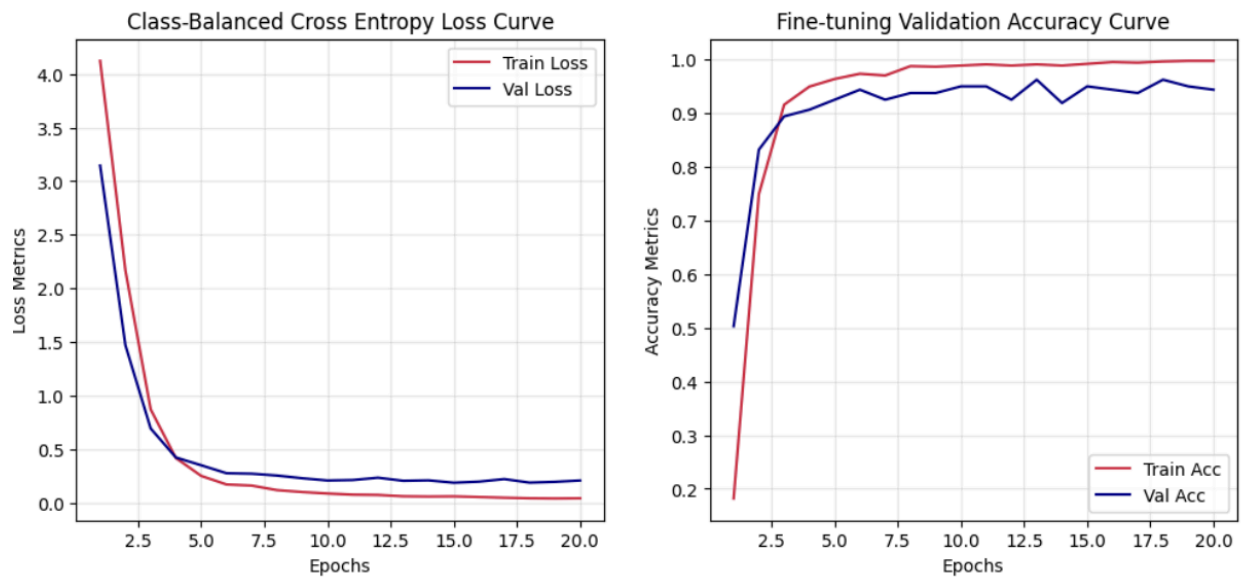
5. Results

- a. Training/validation accuracy (and loss) and any key curves/plots (optional).

- i. final.ipynb (EfficientNet)



- ii. siglipmodel.ipynb (SigLIP2)



b. Kaggle public leaderboard score (and submission settings).

3

gradient



0.96363

6

6h



Your Best Entry!

Your most recent submission scored 0.96363, which is an improvement over your previous score of 0.95454. Great job!

[Tweet this](#)

Submission and Description		Public Score i	Select
	submission4.csv Complete · reagarwa · 6h ago	0.96363	☐
	submission3.csv Complete · reagarwa · 3d ago	0.92727	☐
	submission2.csv Complete · reagarwa · 3d ago	0.95454	☐
	submission1.csv Complete · reagarwa · 3d ago	0.95454	☐
	submission.csv Complete · reagarwa · 5d ago	0.67272	☐
	submission.csv Complete · Leo Li · 6d ago	0.61818	☐

- c. Brief qualitative examples or error analysis highlights (optional).

SigLIP2 (siglipmodel.ipynb) performs stably on the validation set.

EfficientNet (final.ipynb) accuracy is relatively low, indicating that training the linear head alone is insufficient under small data conditions.

Similar categories, low-resolution or occluded samples, may be the main sources of errors

6. Discussion

- a. What worked best and why.

- i. EfficientNet (final.ipynb)

In Phase 1, we froze the convolutional backbone and trained only a

new 100-class linear head for 10 epochs ($lr=1e-3$).

In Phase 2, we unfroze all parameters and trained for 20 more epochs ($lr=1e-4$).

We used 224×224 inputs, ImageNet normalization, and light augmentation (horizontal flip + rotation).

This model reached a best validation accuracy of 58.02%. Training accuracy rose to ~98%, but validation accuracy is around 55–58%, indicating limited transfer with our small dataset and simple fine-tuning.

ii. SigLIP2 (siglipmodel.ipynb) – Worked the best

We used a pretrained ViT-SO400M-14-SigLIP2-378 backbone with WebLI pretrained weights. Instead of fully fine-tuning the entire model, we froze the visual backbone and applied LoRA adapters to the attention qkv layers in the last 4 transformer blocks.

We trained the model for 20 epochs using 378×378 inputs, SigLIP normalization, RandAugment, weighted cross entropy loss, AdamW, and a cosine annealing learning rate scheduler.

This model reached a best validation accuracy of 96.27% and a Kaggle public score of 0.96363, placing us 3rd on the leaderboard. This worked best because SigLIP2 is a much stronger pretrained foundation model than our earlier CNN-based backbones, and LoRA allowed us to adapt the model to our 100-class dataset without overfitting as heavily as

full fine-tuning. The higher input resolution and stronger augmentation also helped the model generalize better from the small training set.

- b. Failure cases, overfitting/underfitting observations.

EfficientNet (final.ipynb): Underfitting on val (train more than 90% vs val around 50%), full fine-tuning has poor generalization on small data.

SigLIP2 (siglipmodel.ipynb): train ~ 98-99% vs val ~ 90-95%, mild overfitting; After epoch 10, val acc was moving around 90-95%, indicating that regularization can be stopped earlier or strengthened.

- c. Limitations and concrete next improvements.

- Small training set
- Did not do hyperparameter tuning (LoRA rank, learning rate, etc.)
- Next improvement:
 - K-fold validation
 - Better data augmentation
 - Early stop
 - hyperparameter tuning

7. Reproducibility

- a. Random seeds and determinism settings.

```
set_seed(42)
```

- b. Package versions/environment setup steps.

torch==2.2.1, open-clip-torch, peft, timm, pandas, matplotlib, pillow, tqdm, numpy, pathlib, PIL, glob, random, torch, torch.nn, [torch.nn.functional](#), torch.utils.data, torchvision, kagglehub.

- c. Exact commands to train and to generate submission.csv.

```
!pip3 install kagglehub
```

```
!pip3 install -q open-clip-torch peft timm pandas matplotlib tqdm pillow
```

Run all the cells.

8. Team Contributions

- a. Reva A

- i. Load the images in matrix form
- ii. Construct a train and validation (evaluate) function
- iii. Training cell in siglipmodel.ipynb
- iv. Plotted the graph

- b. Leo Li

- i. Build up the transformers
- ii. Load the train, validation, and test set
- iii. Phase 1 and 2 training in the final.ipynb

- c. Dan Pham

- i. Turning all parameters
- ii. Make predictions on the model, load into CSV

9. References

- a. <https://replicate.com/lucataco/siglip/readme>

- b. OpenCLIP / SigLIP2: https://github.com/mlfoundations/open_clip
- c. PEFT (LoRA): <https://github.com/huggingface/peft>